

```
=====
FILE: public/config.js
=====
```

```
00001 | (() => {
00002 |   const savedApiBase = localStorage.getItem("campusPulseApiBase") || "";
00003 |   const defaultApiBase = "https://campuspulse-api-ayush.onrender.com";
00004 |   const sameOriginApi =
00005 |     (location.hostname === "localhost" && location.port === "8787") ||
00006 |     location.hostname.endsWith(".onrender.com")
00007 |     ? location.origin
00008 |     : "";
00009 |   window.CAMPUSPULSE_CONFIG = {
00010 |     apiBase:
00011 |       savedApiBase === "offline"
00012 |       ? ""
00013 |       : savedApiBase || sameOriginApi || defaultApiBase,
00014 |   };
00015 | })();
```

```
=====
FILE: public/app.js
=====
```

```
00001 | const APP_VERSION = "1.9.0";
00002 | const API_BASE = String(window.CAMPUSPULSE_CONFIG?.apiBase || "").replace(/\/+$/, "");
00003 | let apiToken = localStorage.getItem("campusPulseApiToken") || "";
00004 |
00005 | // A free-tier API sleeps when idle and takes the better part of a minute to
00006 | // wake, and a whole class opening the app at once arrives while it is still
00007 | // waking. Without a timeout a request could hang indefinitely; without a retry
00008 | // one unlucky moment surfaced to the student as a bare "could not fetch".
00009 | const REQUEST_TIMEOUT_MS = 60000;
00010 | // A sleeping or restarting instance answers with these before any handler runs,
00011 | // so the request provably had no effect and replaying it is safe.
00012 | const UNSERVED_STATUSES = new Set([502, 503, 504]);
00013 | // Safe to replay only for reads, where a repeat cannot change anything even if
00014 | // the first attempt did reach a handler.
00015 | const BUSY_STATUSES = new Set([429, 500, 522, 524]);
00016 | const RETRY_DELAYS_MS = [800, 2000, 4500];
00017 |
00018 | function isOffline() {
00019 |   return typeof navigator !== "undefined" && navigator.onLine === false;
00020 | }
00021 |
00022 | async function fetchOnce(path, options, headers) {
00023 |   // `AbortSignal.timeout` is missing on some older in-app webviews.
00024 |   const controller = new AbortController();
00025 |   const timer = setTimeout(() => controller.abort(), REQUEST_TIMEOUT_MS);
00026 |   try {
00027 |     return await fetch(`${API_BASE}${path}`, {
00028 |       method: options.method || "GET",
00029 |       headers,
00030 |       body: options.body !== undefined ? JSON.stringify(options.body) : undefined,
00031 |       signal: controller.signal,
00032 |     });
00033 |   } finally {
00034 |     clearTimeout(timer);
00035 |   }
00036 | }
00037 |
00038 | async function apiRequest(path, options = {}) {
00039 |   if (!API_BASE) {
00040 |     const error = new Error("Backend API is not configured");
00041 |     error.code = "API_NOT_CONFIGURED";
00042 |     throw error;
00043 |   }
00044 |   const headers = { ...(options.headers || {}) };
00045 |   if (options.body !== undefined) headers["Content-Type"] = "application/json";
00046 |   if (options.auth !== false && apiToken) headers.Authorization = `Bearer ${apiToken}`;
00047 |   const method = options.method || "GET";
00048 |   const isRead = method === "GET" || method === "HEAD";
00049 |   const attempts = options.retry === false ? 1 : RETRY_DELAYS_MS.length + 1;
00050 |
00051 |   let lastError = null;
00052 |   for (let attempt = 0; attempt < attempts; attempt += 1) {
00053 |     if (attempt > 0) {
00054 |       const base = RETRY_DELAYS_MS[attempt - 1];
00055 |       // Jitter keeps a room full of phones from retrying in lockstep.
00056 |       const wait = base + Math.random() * base * 0.5;
00057 |       await new Promise((resolve) => setTimeout(resolve, wait));
00058 |     }
00059 |
00060 |     let response;
00061 |     try {
00062 |       response = await fetchOnce(path, options, headers);
00063 |     } catch (error) {
00064 |       // A dropped connection, a DNS failure, or our own timeout.
00065 |       lastError = new Error(
00066 |         isOffline()
00067 |         ? "You appear to be offline. Reconnect and try again."
00068 |         : "Could not reach CampusPulse. The server may be waking up – trying again.",
00069 |       );
00070 |       lastError.code = "NETWORK";
00071 |     }
```

```
00071 |         lastError.cause = error;
00072 |         continue;
00073 |     }
00074 |
00075 |     const worthRetrying =
00076 |         UNSERVED_STATUSES.has(response.status) ||
00077 |         (isRead && BUSY_STATUSES.has(response.status));
00078 |     if (!response.ok && worthRetrying) {
00079 |         lastError = new Error(
00080 |             response.status === 429
00081 |             ? "CampusPulse is busy right now. Trying again."
00082 |             : "CampusPulse is starting up. Trying again.",
00083 |         );
00084 |         lastError.status = response.status;
00085 |         continue;
00086 |     }
00087 |
00088 |     if (response.status === 204) return null;
00089 |     const contentType = response.headers.get("content-type") || "";
00090 |     const payload = contentType.includes("application/json")
00091 |         ? await response.json()
00092 |         : await response.text();
00093 |     if (!response.ok) {
00094 |         const error = new Error(
00095 |             payload?.error || `Backend request failed (${response.status})`,
00096 |         );
00097 |         error.status = response.status;
00098 |         throw error;
00099 |     }
00100 |     return payload;
00101 | }
00102 |
00103 | const failure =
00104 |     lastError || new Error("Could not reach CampusPulse. Try again in a moment.");
00105 | if (failure.code === "NETWORK" && !isOffline()) {
00106 |     failure.message =
00107 |         "Could not reach CampusPulse after several tries. Check your connection, or the server may still
00108 | be starting up.";
00109 | } else if (failure.status) {
00110 |     failure.message =
00111 |         "CampusPulse is still busy. Wait a few seconds and try again.";
00112 | }
00113 | throw failure;
00114 | }
00115 | async function apiFileUpload(path, file) {
00116 |     const response = await fetch(`${API_BASE}${path}`, {
00117 |         method: "POST",
00118 |         headers: {
00119 |             Authorization: `Bearer ${apiToken}`,
00120 |             "Content-Type": file.type || "application/octet-stream",
00121 |             "X-File-Name": encodeURIComponent(file.name)
00122 |         },
00123 |         body: file
00124 |     });
00125 |     const payload = await response.json().catch(() => ({}));
00126 |     if (!response.ok) {
00127 |         throw new Error(payload.error || `Upload failed (${response.status})`);
00128 |     }
00129 |     return payload;
00130 | }
00131 |
00132 | async function materialBlob(materialId) {
00133 |     const response = await fetch(
00134 |         `${API_BASE}/api/materials/${encodeURIComponent(materialId)}/download`,
00135 |         { headers: { Authorization: `Bearer ${apiToken}` } }
00136 |     );
00137 |     if (!response.ok) {
00138 |         const payload = await response.json().catch(() => ({}));
00139 |         throw new Error(payload.error || `Download failed (${response.status})`);
00140 |     }
00141 |     return response.blob();
00142 | }
00143 |
00144 | function backendConfigured() {
00145 |     return Boolean(API_BASE);
00146 | }
00147 |
00148 | const defaultState = {
00149 |     route: "dashboard",
00150 |     userRole: "faculty",
00151 |     authenticated: false,
00152 |     accountName: "",
00153 |     authEmail: "",
00154 |     accounts: [],
00155 |     backendSchedule: [],
00156 |     backendAttendanceId: "",
00157 |     attendanceCheckedIn: false,
00158 |     backendQuizId: "",
00159 |     backendQuizCourseId: "",
00160 |     backendQuizTitle: "",
00161 |     backendQuizClassLabel: "",
00162 |     backendQuizQuestions: [],
```

```
00163 | quizResponded: false,
00164 | attendanceStatus: "not_started",
00165 | checks: { wifi: false, bluetooth: false },
00166 | quizPublished: false,
00167 | quizResponses: 0,
00168 | courses: [],
00169 | enrolledCourses: [],
00170 | teachingAssistants: [],
00171 | selectedCourseId: "",
00172 | stats: {},
00173 | statsByCourse: {},
00174 | importedSchedule: []
00175 | };
00176 |
00177 | function loadStoredState() {
00178 |   try {
00179 |     return JSON.parse(localStorage.getItem("campusPulseState") || "{}");
00180 |   } catch {
00181 |     localStorage.removeItem("campusPulseState");
00182 |     return {};
00183 |   }
00184 | }
00185 |
00186 | let state = { ...defaultState, ...loadStoredState() };
00187 | state.courses = defaultState.courses;
00188 | state.enrolledCourses = Array.isArray(state.enrolledCourses)
00189 |   ? state.enrolledCourses.filter(courseId => defaultState.courses.some(course => course.id ===
| courseId))
00190 |   : [];
00191 | state.selectedCourseId = String(state.selectedCourseId || "");
00192 | delete state.present;
00193 | state.importedSchedule = Array.isArray(state.importedSchedule) ? state.importedSchedule : [];
00194 | state.backendSchedule = Array.isArray(state.backendSchedule) ? state.backendSchedule : [];
00195 | state.backendQuizQuestions = Array.isArray(state.backendQuizQuestions) ? state.backendQuizQuestions :
| [];
00196 | state.teachingAssistants = Array.isArray(state.teachingAssistants) ? state.teachingAssistants : [];
00197 | state.stats = state.stats && typeof state.stats === "object" ? state.stats : {};
00198 | state.statsByCourse = state.statsByCourse && typeof state.statsByCourse === "object"
00199 |   ? state.statsByCourse
00200 |   : {};
00201 | state.attendanceCheckedIn = Boolean(state.attendanceCheckedIn);
00202 | state.quizResponded = Boolean(state.quizResponded);
00203 | state.accounts = Array.isArray(state.accounts) ? state.accounts.filter(account => account?.email &&
| account?.passwordHash && account?.role) : [];
00204 | state.authenticated = Boolean(state.authenticated);
00205 | if (!state.authenticated) {
00206 |   state.accountName = "";
00207 |   state.authEmail = "";
00208 | }
00209 | let scanTimer;
00210 | let quizTimer;
00211 | let activeAttendance = null;
00212 | // Open sessions a student may mark themselves present in, polled while signed
00213 | // in so the card appears as soon as the professor starts attendance.
00214 | let openAttendance = [];
00215 | let openAttendanceTimer = null;
00216 | let enrolledStudents = [];
00217 | let quizDrafts = [];
00218 | let editingDraftId = "";
00219 | let quizHistory = [];
00220 | let myQuizzes = [];
00221 | let myQuizId = "";
00222 | let attendanceHistory = null;
00223 | let attendanceDayId = "";
00224 | let proximityCode = null;
00225 | let beaconToken = "";
00226 | // Set whenever the native plugin rejects startBeacon, so the UI can show the
00227 | // real reason (e.g. "Turn Bluetooth on") instead of a generic message.
00228 | let beaconError = "";
00229 | let proximityTimer = null;
00230 | // The professor/TA's list of past attendance days for the selected course,
00231 | // and whichever one is currently open for read-only review (null = today's).
00232 | let pastAttendanceSessions = [];
00233 | let pastSessionsLoadedFor = "";
00234 | let viewingPastAttendance = null;
00235 | // The student whose whole record the course team is looking at, if any, and
00236 | // the screen it was opened from.
00237 | let studentRecord = null;
00238 | let studentRecordRoute = "attendance";
00239 | // Which exam a marks spreadsheet is about to be recorded against.
00240 | let pendingMarksUpload = null;
00241 | // What is typed into the search box on the register and the student list.
00242 | let rosterSearch = "";
00243 | let studentSearch = "";
00244 | // The marks grid for the selected course, so totals can be shown and prefilled.
00245 | let courseMarks = null;
00246 | let courseMarksLoadedFor = "";
00247 |
00248 | // Matches a student by name or roll number, ignoring case, spacing and the
00249 | // punctuation that creeps into a roll number when it is copied around.
00250 | function matchesSearch(student, query) {
00251 |   const needle = String(query || "").trim().toLowerCase();
00252 |   if (!needle) return true;
```

```
00253 |   const name = String(student?.name || "").toLowerCase();
00254 |   const roll = String(student?.rollNumber || "").toLowerCase();
00255 |   if (name.includes(needle) || roll.includes(needle)) return true;
00256 |   const loose = (value) => value.replace(/^[a-z0-9]/g, "");
00257 |   return loose(roll).includes(loose(needle));
00258 | }
00259 |
00260 | function searchBox(id, value, placeholder) {
00261 |   return `<label class="roster-search">
00262 |     <span class="visually-hidden">${escapeHtml(placeholder)}</span>
00263 |     <input class="text-input" id="${id}" type="search" value="${escapeHtml(value)}"
00264 |       placeholder="${escapeHtml(placeholder)}" autocomplete="off" data-search-box />
00265 |   </label>`;
00266 | }
00267 | // Rows being edited in the exams card, held separately from the saved list so
00268 | // an unsaved addition survives a re-render.
00269 | let examDraft = null;
00270 |
00271 | function examEditorRows() {
00272 |   if (examDraft) return examDraft;
00273 |   return (courseMarks?.exams || EXAM_CHOICES.map(exam => ({ ...exam, maxMarks: null })))
00274 |     .map(exam => ({ id: exam.id, label: exam.label, maxMarks: exam.maxMarks ?? null }));
00275 | }
00276 |
00277 | // Reads whatever is currently typed into the card, so nothing is lost when a
00278 | // row is added or removed.
00279 | function readExamRows() {
00280 |   return [...view.querySelectorAll("[data-exam-row]")].map(row => ({
00281 |     id: row.querySelector("[data-exam-id]")?.value || "",
00282 |     label: row.querySelector("[data-exam-label]")?.value || "",
00283 |     maxMarks: row.querySelector("[data-exam-max]")?.value.trim() || null
00284 |   }));
00285 | }
00286 |
00287 | const EXAM_CHOICES = [
00288 |   { id: "test1", label: "Test 1" },
00289 |   { id: "test2", label: "Test 2" },
00290 |   { id: "test3", label: "Test 3" },
00291 |   { id: "test4", label: "Test 4" },
00292 |   { id: "test5", label: "Test 5" },
00293 |   { id: "test6", label: "Test 6" },
00294 |   { id: "midsem", label: "Mid Sem" },
00295 |   { id: "endsem", label: "End Sem" }
00296 | ];
00297 | let courseNotices = [];
00298 | let quizResults = null;
00299 | let pendingSignup = null;
00300 | let passwordResetEmail = "";
00301 | // Reset by email is only offered when the server can actually send one.
00302 | let emailDeliveryAvailable = false;
00303 |
00304 | async function refreshEmailDeliveryState() {
00305 |   if (!backendConfigured()) return;
00306 |   try {
00307 |     const health = await apiRequest("/api/health", { auth: false });
00308 |     const available = Boolean(health.emailDelivery) && health.emailDelivery !== "disabled";
00309 |     if (available !== emailDeliveryAvailable) {
00310 |       emailDeliveryAvailable = available;
00311 |       // Never redraw over someone part-way through entering their code.
00312 |       if (!state.authenticated && !pendingSignup) {
00313 |         renderLogin(selectedLoginRole, authMode);
00314 |       }
00315 |     }
00316 |   } catch {
00317 |     // Sign-in must still work when the health probe cannot be reached.
00318 |   }
00319 | }
00320 | let courseRosters = new Map();
00321 | let managedCourseId = "";
00322 | let courseMaterials = new Map();
00323 | let materialsCourseId = "";
00324 | let editingScheduleIndex = -1;
00325 | let modalReturnFocus = null;
00326 | let selectedLoginRole = state.userRole || "faculty";
00327 | let authMode = state.accounts.length ? "login" : "signup";
00328 | const view = document.querySelector("#view");
00329 | const authRoot = document.querySelector("#authRoot");
00330 | const appShell = document.querySelector("#appShell");
00331 | const pageTitle = document.querySelector("#pageTitle");
00332 | const pageEyebrow = document.querySelector("#pageEyebrow");
00333 | const quickAction = document.querySelector("#quickAction");
00334 | const courseSwitcher = document.querySelector("#courseSwitcher");
00335 | const courseSwitcherWrap = document.querySelector("#courseSwitcherWrap");
00336 | const profileAvatar = document.querySelector("#profileAvatar");
00337 | const profileName = document.querySelector("#profileName");
00338 | const profileMeta = document.querySelector("#profileMeta");
00339 | const updateBanner = document.querySelector("#updateBanner");
00340 | const updateBannerMessage = document.querySelector("#updateBannerMessage");
00341 | const updateManager = window.CAMPUSPULSE_UPDATES || null;
00342 | const reminderManager = window.CAMPUSPULSE_REMINDERS || null;
00343 | const pushManager = window.CAMPUSPULSE_PUSH || null;
00344 | const notificationButton = document.querySelector("#notificationButton");
00345 | const notificationBadge = document.querySelector("#notificationBadge");
```

```
00346 | const appMenu = document.querySelector("#appMenu");
00347 | const menuScrim = document.querySelector("#menuScrim");
00348 | const menuToggle = document.querySelector("#menuToggle");
00349 | const menuAvatar = document.querySelector("#menuAvatar");
00350 | const menuName = document.querySelector("#menuName");
00351 | const menuMeta = document.querySelector("#menuMeta");
00352 |
00353 | function setMenuOpen(open) {
00354 |   if (!appMenu) return;
00355 |   appMenu.classList.toggle("open", open);
00356 |   appMenu.setAttribute("aria-hidden", open ? "false" : "true");
00357 |   menuScrim?.classList.toggle("open", open);
00358 |   menuToggle?.setAttribute("aria-expanded", open ? "true" : "false");
00359 |   if (open) appMenu.querySelector(".app-menu-item)?.focus();
00360 |   else if (document.activeElement && appMenu.contains(document.activeElement)) {
00361 |     menuToggle?.focus();
00362 |   }
00363 | }
00364 |
00365 | function closeMenu() {
00366 |   setMenuOpen(false);
00367 | }
00368 |
00369 | menuToggle?.addEventListener("click", () => setMenuOpen(!appMenu.classList.contains("open")));
00370 | menuScrim?.addEventListener("click", closeMenu);
00371 | document.querySelector("#menuClose)?.addEventListener("click", closeMenu);
00372 | appMenu?.addEventListener("click", event => {
00373 |   if (event.target.closest(".app-menu-item")) closeMenu();
00374 | });
00375 | document.addEventListener("keydown", event => {
00376 |   if (event.key === "Escape" && appMenu?.classList.contains("open")) closeMenu();
00377 | });
00378 |
00379 | function updateStatusLabel(status) {
00380 |   return {
00381 |     checking: "Checking",
00382 |     downloading: "Downloading",
00383 |     ready: "Installs shortly",
00384 |     applying: "Applying",
00385 |     current: "Up to date",
00386 |     error: "Embedded fallback",
00387 |     idle: "Automatic",
00388 |     unavailable: "Web deployment",
00389 |   }[status] || "Automatic";
00390 | }
00391 |
00392 | function syncUpdateUi(updateState = updateManager?.state || {}) {
00393 |   if (updateBanner) {
00394 |     updateBanner.hidden = updateState.status !== "ready";
00395 |     if (updateBannerMessage && updateState.message) {
00396 |       updateBannerMessage.textContent = updateState.message;
00397 |     }
00398 |   }
00399 |   const status = document.querySelector("#webUpdateStatus");
00400 |   const detail = document.querySelector("#webUpdateDetail");
00401 |   if (status) status.textContent = updateStatusLabel(updateState.status);
00402 |   if (detail && updateState.message) detail.textContent = updateState.message;
00403 | }
00404 |
00405 | updateManager?.subscribe(syncUpdateUi);
00406 |
00407 | // A downloaded bundle reloads the webview, so hold it back while a class is
00408 | // actually running or a dialog is open, and apply it on the next screen change.
00409 | function updatesAreSafeToApply() {
00410 |   const attendanceLive =
00411 |     activeAttendance?.status === "open" || state.attendanceStatus === "scanning";
00412 |   const modalOpen = Boolean(document.querySelector("#modalRoot)?.innerHTML.trim());
00413 |   return !attendanceLive && !modalOpen;
00414 | }
00415 |
00416 | updateManager?.setApplyGuard?.(updatesAreSafeToApply);
00417 |
00418 | function icon(id) {
00419 |   return `<svg aria-hidden="true"><use href="#${id}"/></svg>`;
00420 | }
00421 |
00422 | function escapeHtml(value = "") {
00423 |   return String(value)
00424 |     .replaceAll("&", "&amp;")
00425 |     .replaceAll("<", "&lt;")
00426 |     .replaceAll(">", "&gt;")
00427 |     .replaceAll("'", "&quot;")
00428 |     .replaceAll('"', "&#039;");
00429 | }
00430 |
00431 | function selectedCourse() {
00432 |   return state.courses.find(course => course.id === state.selectedCourseId) || state.courses[0] || null;
00433 | }
00434 |
00435 | function softComputingCourse() {
00436 |   return selectedCourse();
00437 | }
00438 |
```

```
00439 | function courseCapabilities(course = selectedCourse()) {
00440 |   return course?.capabilities || {};
00441 | }
00442 |
00443 | function canManageCourse(course = selectedCourse()) {
00444 |   return Boolean(courseCapabilities(course).canManageCourse);
00445 | }
00446 |
00447 | function canManageSchedule(course = selectedCourse()) {
00448 |   return Boolean(courseCapabilities(course).canManageSchedule);
00449 | }
00450 |
00451 | function canManageRoster(course = selectedCourse()) {
00452 |   return Boolean(courseCapabilities(course).canManageRoster);
00453 | }
00454 |
00455 | function canRunAttendance(course = selectedCourse()) {
00456 |   return Boolean(courseCapabilities(course).canRunAttendance);
00457 | }
00458 |
00459 | function canPublishQuiz(course = selectedCourse()) {
00460 |   return Boolean(courseCapabilities(course).canPublishQuiz);
00461 | }
00462 |
00463 | function canUploadMaterials(course = selectedCourse()) {
00464 |   return Boolean(courseCapabilities(course).canUploadMaterials);
00465 | }
00466 |
00467 | function attendanceCourse() {
00468 |   return state.courses.find(course => course.id === activeAttendance?.courseId) || selectedCourse();
00469 | }
00470 |
00471 | async function loadCourseRoster(courseId, { force = false } = {}) {
00472 |   const course = state.courses.find(item => item.id === courseId);
00473 |   if (!courseCapabilities(course).canViewAttendanceRoster) {
00474 |     throw new Error("You do not have roster access for this course");
00475 |   }
00476 |   if (!force && courseRosters.has(courseId)) return courseRosters.get(courseId);
00477 |   const result = await apiRequest(`/api/courses/${encodeURIComponent(courseId)}/roster`);
00478 |   const roster = result.students || [];
00479 |   courseRosters.set(courseId, roster);
00480 |   return roster;
00481 | }
00482 |
00483 | async function loadCourseMaterials(courseId, { force = false } = {}) {
00484 |   if (!force && courseMaterials.has(courseId)) return courseMaterials.get(courseId);
00485 |   const result = await apiRequest(
00486 |     `/api/courses/${encodeURIComponent(courseId)}/materials`
00487 |   );
00488 |   const materials = result.materials || [];
00489 |   courseMaterials.set(courseId, materials);
00490 |   return materials;
00491 | }
00492 |
00493 | function currentAttendanceRecords() {
00494 |   return Array.isArray(activeAttendance?.records) ? activeAttendance.records : [];
00495 | }
00496 |
00497 | function currentPresentCount() {
00498 |   return currentAttendanceRecords().filter(record => record.present).length;
00499 | }
00500 |
00501 | function applyAttendanceSnapshot(attendance) {
00502 |   activeAttendance = attendance || null;
00503 |   state.backendAttendanceId = attendance?.id || "";
00504 |   state.attendanceStatus = attendance
00505 |     ? attendance.status === "open" ? "scanning" : "complete"
00506 |     : "not_started";
00507 |   if (attendance?.courseId) state.selectedCourseId = attendance.courseId;
00508 |   // A newly applied snapshot is always today's session, not a past one being
00509 |   // browsed, and not a student's record opened from some other course.
00510 |   viewingPastAttendance = null;
00511 |   studentRecord = null;
00512 | }
00513 |
00514 | async function selectAttendanceCourse(courseId) {
00515 |   let course = state.courses.find(item => item.id === courseId);
00516 |   if (!course) {
00517 |     try { await syncBackendState(); } catch { /* best effort */ }
00518 |     course = state.courses.find(item => item.id === courseId);
00519 |   }
00520 |   if (!course) throw new Error("Course not found");
00521 |   if (!canRunAttendance(course)) throw new Error("You cannot run attendance for this course");
00522 |   state.selectedCourseId = course.id;
00523 |   if (activeAttendance?.courseId === course.id) return;
00524 |   if (!backendConfigured() || !apiKey) {
00525 |     applyAttendanceSnapshot(null);
00526 |     return;
00527 |   }
00528 |   const payload = await apiRequest(
00529 |     `/api/attendance/current?courseId=${encodeURIComponent(course.id)}`
00530 |   );
00531 |   applyAttendanceSnapshot(payload.attendance);
00532 | }
```



```
00532 | }
00533 |
00534 | // Lazily loaded once per course so switching back to "today" doesn't refetch.
00535 | async function refreshPastSessions(courseId) {
00536 |   if (!backendConfigured() || !apiToken || !courseId) {
00537 |     pastAttendanceSessions = [];
00538 |     pastSessionsLoadedFor = "";
00539 |     return;
00540 |   }
00541 |   pastSessionsLoadedFor = courseId;
00542 |   try {
00543 |     const result = await apiRequest(`/api/attendance/past?courseId=${encodeURIComponent(courseId)}`);
00544 |     pastAttendanceSessions = result.sessions || [];
00545 |   } catch {
00546 |     pastAttendanceSessions = [];
00547 |     pastSessionsLoadedFor = "";
00548 |   }
00549 | }
00550 |
00551 | // The history picker belongs on the attendance page at all times, not only
00552 | // while a session happens to be open: the commonest reason to open attendance
00553 | // on a quiet day is to look up what a past class recorded.
00554 | function pastSessionsPicker() {
00555 |   const current = viewingPastAttendance?.id || "";
00556 |   if (!pastAttendanceSessions.length) {
00557 |     return `
```

```
00618 | class="stat-value">${average}%</div><div class="stat-label">Average attendance</div></article>
00619 | <article class="card stat"><div class="stat-top"><span class="stat-icon
00620 | amber">${icon("i-clock")}</span></div><div class="stat-value">${lowest}%</div><div
00621 | class="stat-label">Lowest turnout</div></article>
00622 | </div>
00623 | </article>`;
00624 | }
00625 | // The dropdown is right for switching classes while a register is on screen,
00626 | // but on the setup page the past classes are the reason to be there at all, so
00627 | // they are listed instead of hidden inside a select.
00628 | function pastSessionsListCard() {
00629 |   if (!pastAttendanceSessions.length) {
00630 |     return `<article class="card page-card">
00631 |       <div class="section-head"><div><h2 style="margin:0 0 5px">Previous classes</h2><p
00632 | class="stat-label">Registers already taken for this course.</p></div><span class="badge
00633 | gray">0</span></div>
00634 | <p class="stat-label">No class recorded yet. Each register appears here once you close it.</p>
00635 | </article>`;
00636 |   }
00637 |   return `<article class="card page-card">
00638 |     <div class="section-head"><div><h2 style="margin:0 0 5px">Previous classes</h2><p
00639 | class="stat-label">Open one to see who was there.</p></div><span class="badge
00640 | gray">${pastAttendanceSessions.length}</span></div>
00641 |     <div class="class-list">
00642 |       ${pastAttendanceSessions.map(session => {
00643 |         const started = new Date(session.startedAt);
00644 |         const share = sessionTurnout(session);
00645 |         // The register already on screen is marked, so the list reads as a
00646 |         // place in the term rather than a set of identical-looking links.
00647 |         const open = viewingPastAttendance?.id === session.id;
00648 |         return `<button class="class-row attendance-day" type="button" data-action="open-past-session"
00649 | data-session-id="${escapeHtml(session.id)}">${open ? ' aria-current="true" : ""}>
00650 | <div class="time">${escapeHtml(started.toLocaleDateString([], { weekday: "short"
00651 | })))<small>${escapeHtml(started.toLocaleDateString([], { day: "numeric", month: "short"
00652 | })))</small></div>
00653 | <div class="course"><strong>${escapeHtml(session.classLabel || (session.importedAt ? "Paper
00654 | register" : started.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit"
00655 | })))}</strong><span>${session.present}/${session.total}</span> present</span></div>
00656 | <span class="badge ${open ? "purple" : turnoutTone(share)}">${open ? "Viewing" :
00657 | `${share}%`}</span>
00658 | <span class="chevron">${icon("i-arrow")}</span>
00659 | </button>`;
00660 |       }).join("")}
00661 |     </div>
00662 |   </article>`;
00663 | }
00664 | // One student's whole record in a course: every class held, whether they were
00665 | // there, and the running percentage. Available to the course team only.
00666 | async function openStudentRecord(courseId, rollNumber) {
00667 |   try {
00668 |     studentRecord = await apiRequest(
00669 |       `/api/courses/${encodeURIComponent(courseId)}/students/${encodeURIComponent(rollNumber)}`
00670 |     );
00671 |     // Remembered so closing the record returns to wherever it was opened from,
00672 |     // whether that was the register or the student list.
00673 |     studentRecordRoute = state.route;
00674 |   } catch (error) {
00675 |     studentRecord = null;
00676 |     return toast(error.message || "Could not open that student's record", "error");
00677 |   }
00678 |   render();
00679 | }
00680 | function renderStudentRecord() {
00681 |   const { student, summary, sessions } = studentRecord;
00682 |   const percentage = summary.percentage;
00683 |   // Only exams that have both a total and a mark can be added up; the rest are
00684 |   // still shown so a mark can be typed in.
00685 |   const marksTotal = (studentRecord.marks || []).reduce(
00686 |     (running, exam) =>
00687 |       exam.maxMarks && exam.score !== null
00688 |         ? { scored: running.scored + exam.score, outOf: running.outOf + exam.maxMarks }
00689 |         : running,
00690 |     { scored: 0, outOf: 0 }
00691 |   );
00692 |   const detail = (label, value) =>
00693 |     value ? `<div class="summary-item"><span>${label}</span><strong>${escapeHtml(value)}</strong></div>`
00694 |     : "";
00695 |   setHeader(student.name || student.rollNumber, `${student.courseCode} · STUDENT RECORD`, false);
00696 |   view.innerHTML = `
00697 |     <button class="back-btn" type="button" data-action="close-student-record">${icon("i-back")} Back to
00698 | attendance</button>
00699 |     <div class="page-grid">
00700 |       <article class="card page-card">
00701 |         <div class="section-head">
00702 |           <div>
00703 |             <h2 style="margin:0 0 5px">${escapeHtml(student.name || "Unnamed student")}</h2>
00704 |             <p class="stat-label">${escapeHtml(student.rollNumber)}>${student.serial ? ` · No.
00705 | ${student.serial}` : ""} · ${escapeHtml(student.courseCode)}</p>
00706 |           </div>
00707 |           <span class="badge ${percentage >= 75 ? "green" : percentage >= 50 ? "amber" :`
```



```
00695 | "red">${percentage}%</span>
00696 | </div>
00697 | <div class="stat-grid" style="margin-top:18px">
00698 | <article class="card stat"><div class="stat-top"><span class="stat-icon
| green">${icon("i-check")}</span></div><div class="stat-value">${summary.attended}</div><div
| class="stat-label">Present</div></article>
00698 | <article class="card stat"><div class="stat-top"><span class="stat-icon
| amber">${icon("i-clock")}</span></div><div class="stat-value">${summary.missed}</div><div
| class="stat-label">Absent</div></article>
00699 | <article class="card stat"><div class="stat-top"><span
| class="stat-icon">${icon("i-calendar")}</span></div><div class="stat-value">${summary.held}</div><div
| class="stat-label">Classes held</div></article>
00700 | </div>
00701 | ${student.hasAccount
00702 | ? `<div class="summary-list" style="margin-top:16px">
00703 |   ${detail("Email", student.email)}
00704 |   ${detail("Department", student.department)}
00705 |   ${detail("Phone", student.phone)}
00706 |   ${detail("Hall", student.hall)}
00707 |   ${detail("Joined", student.joinedAt ? new Date(student.joinedAt).toLocaleDateString() :
| "")}
00708 | </div>`
00709 | : `<div class="security-note" style="margin-top:16px"><span class="lock">■</span><span>On the
| roll list, but has not signed up yet. Contact details appear once they create an account.</span></div>`}
00710 | <div class="setup-actions" style="margin-top:16px">
00711 | <button class="btn btn-soft" type="button"
| data-action="export-student-record">${icon("i-download")} Download record</button>
00712 | </div>
00713 | </article>
00714 | <article class="card page-card">
00715 | <div class="section-head"><div><h2 style="margin:0 0 5px">Exam marks</h2><p
| class="stat-label">Type a mark and press Enter, or leave it blank if they did not sit the
| exam.</p></div><span class="badge ${marksTotal.outOf ? "purple" : "gray"}">${marksTotal.outOf ?
| `${marksTotal.scored}/${marksTotal.outOf}` : "Not set up"}</span></div>
00716 | <div class="roster-scroll">
00717 | <table class="roster-table marks-table">
00718 | <thead><tr><th>Exam</th><th>Out of</th><th>Marks</th></tr></thead>
00719 | <tbody>${(studentRecord.marks || []).map(exam => `<tr>
00720 | <td>${escapeHtml(exam.label)}</td>
00721 | <td class="roster-roll">${exam.maxMarks ?? "-"}</td>
00722 | <td>
00723 | <input class="text-input marks-input" type="number" min="0" step="0.5"
00724 |   ${exam.maxMarks ? `max="${exam.maxMarks}"` : ""}
00725 |   value="${exam.score ?? ""}"
00726 |   data-mark-for="${escapeHtml(exam.id)}"
00727 |   aria-label="${escapeHtml(exam.label)} marks"
00728 |   placeholder="-" />
00729 | </td>
00730 | </tr>`).join("")}</tbody>
00731 | </table>
00732 | </div>
00733 | <div class="setup-actions" style="margin-top:14px">
00734 | <button class="btn btn-primary" type="button"
| data-action="save-student-marks">${icon("i-check")} Save marks</button>
00735 | </div>
00736 | </article>
00737 | <article class="card page-card">
00738 | <div class="section-head"><div><h2 style="margin:0 0 5px">Every class</h2><p
| class="stat-label">Newest first. Classes held before this student joined the roll list are not
| counted.</p></div><span class="badge gray">${sessions.length}</span></div>
00739 | ${sessions.length ? `<div class="class-list">
00740 |   ${sessions.map(session => {
00741 |     const when = new Date(session.startedAt);
00742 |     return `<div class="class-row">
00743 |       <div class="time">${escapeHtml(when.toLocaleDateString([], { weekday: "short"
| })))}<small>${escapeHtml(when.toLocaleDateString([], { day: "numeric", month: "short" })))}</small></div>
00744 |       <div class="course"><strong>${escapeHtml(session.classLabel || (session.importedAt ?
| "Paper register" : when.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit"
| })))}</strong><span>${escapeHtml(session.room || "Room TBA")}</span>${session.present && session.markedVia ===
| "student" ? " " · marked over Bluetooth" : ""}</span></div>
00745 |       <span class="badge ${session.present ? "green" : "gray"}">${session.present ? "Present" :
| "Absent"}</span>
00746 |     </div>`;
00747 |   }).join("")}
00748 | </div>` : `<p class="stat-label">No attendance has been taken for this course yet.</p>`}
00749 | </article>
00750 | </div>`;
00751 | }
00752 |
00753 | // Loads a past (closed) session's full roster for read-only review, or clears
00754 | // back to today's session when the dropdown is reset to its default option.
00755 | async function openPastAttendanceSession(sessionId) {
00756 |   // Clearing the picker returns to whatever the page would otherwise show:
00757 |   // today's session if one exists, the setup screen if not. Going straight to
00758 |   // the live view would leave an empty roster on a day with no session.
00759 |   if (!sessionId) {
00760 |     viewingPastAttendance = null;
00761 |     return renderAttendance();
00762 |   }
00763 |   try {
00764 |     const result = await apiRequest(`api/attendance/${encodeURIComponent(sessionId)}`);
00765 |     viewingPastAttendance = result.attendance;
00766 |   } catch (error) {
```

```
00767 |     viewingPastAttendance = null;
00768 |     return toast(error.message || "Could not load that session", "error");
00769 | }
00770 | renderAttendance();
00771 | }
00772 |
00773 | function applyQuizSnapshot(quiz) {
00774 |     state.backendQuizId = quiz?.id || "";
00775 |     state.backendQuizCourseId = quiz?.courseId || "";
00776 |     state.backendQuizTitle = quiz?.title || "";
00777 |     state.backendQuizClassLabel = quiz?.classLabel || quiz?.day || "";
00778 |     state.backendQuizQuestions = quiz?.questions || [];
00779 |     state.quizPublished = quiz?.status === "open";
00780 |     state.quizResponses = Array.isArray(quiz?.responses) ? quiz.responses.length : 0;
00781 |     state.quizResponded = Boolean(quiz?.responded);
00782 | }
00783 |
00784 | async function selectQuizCourse(courseId) {
00785 |     let course = state.courses.find(item => item.id === courseId);
00786 |     if (!course) {
00787 |         try { await syncBackendState(); } catch { /* best effort */ }
00788 |         course = state.courses.find(item => item.id === courseId);
00789 |     }
00790 |     if (!course) throw new Error("Course not found");
00791 |     if (state.userRole !== "student" && !canPublishQuiz(course)) {
00792 |         throw new Error("You cannot publish quizzes for this course");
00793 |     }
00794 |     state.selectedCourseId = course.id;
00795 |     if (!backendConfigured() || !apiToken) {
00796 |         applyQuizSnapshot(null);
00797 |         return;
00798 |     }
00799 |     const payload = await apiRequest(
00800 |         `/api/quizzes/current?courseId=${encodeURIComponent(course.id)}`
00801 |     );
00802 |     applyQuizSnapshot(payload.quiz);
00803 | }
00804 |
00805 | function clearSensitiveClientState({ clearImportedSchedule = false } = {}) {
00806 |     clearInterval(scanTimer);
00807 |     clearTimeout(quizTimer);
00808 |     clearInterval(openAttendanceTimer);
00809 |     clearInterval(proximityTimer);
00810 |     proximityCode = null;
00811 |     stopAttendanceBeacon();
00812 |     attendanceHistory = null;
00813 |     attendanceDayId = "";
00814 |     myQuizzes = [];
00815 |     myQuizId = "";
00816 |     openAttendance = [];
00817 |     enrolledStudents = [];
00818 |     quizDrafts = [];
00819 |     courseRosters = new Map();
00820 |     courseMaterials = new Map();
00821 |     activeAttendance = null;
00822 |     managedCourseId = "";
00823 |     materialsCourseId = "";
00824 |     state.backendAttendanceId = "";
00825 |     state.attendanceStatus = "not_started";
00826 |     state.attendanceCheckedIn = false;
00827 |     state.backendQuizId = "";
00828 |     state.backendQuizCourseId = "";
00829 |     state.backendQuizTitle = "";
00830 |     state.backendQuizClassLabel = "";
00831 |     state.backendQuizQuestions = [];
00832 |     state.quizPublished = false;
00833 |     state.quizResponded = false;
00834 |     state.courses = defaultState.courses;
00835 |     state.enrolledCourses = [];
00836 |     state.teachingAssistants = [];
00837 |     state.selectedCourseId = "";
00838 |     state.stats = {};
00839 |     state.statsByCourse = {};
00840 |     state.backendSchedule = [];
00841 |     if (clearImportedSchedule) state.importedSchedule = [];
00842 |     if (view) view.innerHTML = "";
00843 | }
00844 |
00845 | function persist() {
00846 |     state.appVersion = APP_VERSION;
00847 |     localStorage.setItem("campusPulseState", JSON.stringify(state));
00848 | }
00849 |
00850 | // The browser's own `pattern` check, kept deliberately permissive so it never
00851 | // rejects an address the server would accept.
00852 | const EMAIL_INPUT_PATTERN =
00853 |     "[A-Za-z0-9][A-Za-z0-9._%+\-]*@[A-Za-z0-9](?:[A-Za-z0-9\-\]{0,61}[A-Za-z0-9])?(?:\.[A-Za-z0-9](?:[A-
00854 |     Za-z0-9\-\]{0,61}[A-Za-z0-9])?)*\.[A-Za-z]{2,63}";
00855 |
00856 | const loginProfiles = {
00857 |     faculty: {
00858 |         title: "Professor login",
00859 |         shortTitle: "Professor",
```

```
00030 |   return {
00031 |     ...data,
00032 |     courseMaterials: (data.courseMaterials || []).map(
00033 |       ({ dataBase64: _bytes, ...metadata }) => metadata,
00034 |     ),
00035 |   };
00036 | }
00037 |
00038 | function createPostgresStore(connectionString, options = {}) {
00039 |   const env = options.env || process.env;
00040 |   const transactionAttempts = Math.max(
00041 |     1,
00042 |     Number(options.transactionAttempts || 5) || 5,
00043 |   );
00044 |   const pool =
00045 |     options.pool ||
00046 |     new Pool({
00047 |       connectionString,
00048 |       ssl: options.ssl ? { rejectUnauthorized: false } : undefined,
00049 |       // Five is enough for this single-document store: writes are serialized
00050 |       // and concurrent reads already share one in-flight query.
00051 |       max: Number(options.maxConnections || 5),
00052 |       idleTimeoutMillis: 30000,
00053 |       connectionTimeoutMillis: 10000,
00054 |     });
00055 |   let ready;
00056 |   // The document last read from Postgres, reused until a write moves the
00057 |   // revision on. A sign-in storm then costs one load instead of one per user.
00058 |   let cache = null;
00059 |   let inFlightRead = null;
00060 |
00061 |   function ensureTable() {
00062 |     if (!ready) {
00063 |       ready = (async () => {
00064 |         await pool.query(`
00065 |           CREATE TABLE IF NOT EXISTS campuspulse_store (
00066 |             id INTEGER PRIMARY KEY,
00067 |             data JSONB NOT NULL,
00068 |             updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
00069 |           )
00070 |         `);
00071 |         // Compared far more cheaply than the document itself, and unlike a
00072 |         // timestamp it always moves for two writes in the same instant.
00073 |         await pool.query(
00074 |           "ALTER TABLE campuspulse_store ADD COLUMN IF NOT EXISTS revision BIGINT NOT NULL DEFAULT 0",
00075 |         );
00076 |       })().catch((error) => {
00077 |         ready = undefined;
00078 |         throw error;
00079 |       });
00080 |     }
00081 |     return ready;
00082 |   }
00083 |
00084 |   async function currentRevision() {
00085 |     const result = await pool.query(
00086 |       "SELECT revision FROM campuspulse_store WHERE id = 1",
00087 |     );
00088 |     return result.rows[0] ? Number(result.rows[0].revision) : null;
00089 |   }
00090 |
00091 |   async function loadFresh() {
00092 |     const result = await pool.query(READ_PROJECTION);
00093 |     const row = result.rows[0];
00094 |     const source = row?.data || initialData(env);
00095 |     const data = normalizeData(source, env);
00096 |     if (row && courseJoinCodesNeedPersistence(source, data)) {
00097 |       // Generated legacy TA codes must survive the response that first revealed
00098 |       // them. `data` has material bytes projected out, so the stored blobs are
00099 |       // carried across rather than overwritten with nothing.
00100 |       const migrated = await pool.query(
00101 |         `UPDATE campuspulse_store
00102 |           SET data = jsonb_set(
00103 |             $1::jsonb,
00104 |             '{courseMaterials}',
00105 |             COALESCE(data->'courseMaterials', '[]'::jsonb)
00106 |           ),
00107 |           revision = revision + 1,
00108 |           updated_at = NOW()
00109 |           WHERE id = 1
00110 |           RETURNING revision`,
00111 |         [JSON.stringify({ ...data, courseMaterials: [] })],
00112 |       );
00113 |       return {
00114 |         data,
00115 |         revision: migrated.rows[0] ? Number(migrated.rows[0].revision) : null,
00116 |       };
00117 |     }
00118 |     // `data` and `revision` come from the same SELECT. Fetching the revision in
00119 |     // a second query allowed a concurrent write to pair old data with its new
00120 |     // revision and poison the cache.
00121 |     return { data, revision: row ? Number(row.revision) : null };
00122 |   }
```

```
00123 |
00124 | async function readSnapshot() {
00125 |     await ensureTable();
00126 |     const cached = cache;
00127 |     if (cached) {
00128 |         const revision = await currentRevision();
00129 |         // A writer may replace or clear the module-level cache while the query is
00130 |         // in flight. The local reference remains valid and must be the one read.
00131 |         if (revision !== null && revision === cached.revision) return cached.data;
00132 |     }
00133 |     const fresh = await loadFresh();
00134 |     const latest = cache;
00135 |     if (
00136 |         !latest ||
00137 |         latest.revision === null ||
00138 |         (fresh.revision !== null && fresh.revision >= latest.revision)
00139 |     ) {
00140 |         cache = fresh;
00141 |         return fresh.data;
00142 |     }
00143 |     // A concurrent writer installed a newer snapshot while this read was in
00144 |     // flight. Return it instead of replacing it with stale data.
00145 |     return latest.data;
00146 | }
00147 |
00148 | const runUpdates = createBatchingUpdater(async (apply) => {
00149 |     await ensureTable();
00150 |     for (let attempt = 0; attempt < transactionAttempts; attempt += 1) {
00151 |         const client = await pool.connect();
00152 |         try {
00153 |             await client.query("BEGIN");
00154 |             await client.query(
00155 |                 `INSERT INTO campuspulse_store (id, data)
00156 |                 VALUES (1, $1::jsonb)
00157 |                 ON CONFLICT (id) DO NOTHING`,
00158 |                 [JSON.stringify(initialData(env))],
00159 |             );
00160 |             const result = await client.query(
00161 |                 "SELECT data FROM campuspulse_store WHERE id = 1 FOR UPDATE",
00162 |             );
00163 |             const data = normalizeData(result.rows[0].data, env);
00164 |             const outcome = await apply(data);
00165 |             const saved = await client.query(
00166 |                 `UPDATE campuspulse_store
00167 |                 SET data = $1::jsonb, revision = revision + 1, updated_at = NOW()
00168 |                 WHERE id = 1
00169 |                 RETURNING revision`,
00170 |                 [JSON.stringify(data)],
00171 |             );
00172 |             await client.query("COMMIT");
00173 |             cache = {
00174 |                 data: withoutMaterialBytes(data),
00175 |                 revision: Number(saved.rows[0].revision),
00176 |             };
00177 |             return outcome;
00178 |         } catch (error) {
00179 |             await client.query("ROLLBACK").catch(() => {});
00180 |             const retryable = error?.code === "40001";
00181 |             if (!retryable || attempt + 1 >= transactionAttempts) {
00182 |                 cache = null;
00183 |                 throw error;
00184 |             }
00185 |         } finally {
00186 |             client.release();
00187 |         }
00188 |         // CockroachDB uses SQLSTATE 40001 when a serializable transaction must
00189 |         // restart. Jitter prevents two contenders from colliding again together.
00190 |         const delay = 10 * 2 ** attempt + Math.floor(Math.random() * 10);
00191 |         await new Promise((resolve) => setTimeout(resolve, delay));
00192 |     }
00193 |     throw new Error("Database transaction retry limit reached");
00194 | });
00195 |
00196 | return {
00197 |     read() {
00198 |         // Callers arriving together share one load instead of queueing behind one
00199 |         // another. The snapshot is read-only by contract and is safe to share;
00200 |         // cloning the full document once per caller let a retrying classroom
00201 |         // allocate hundreds of copies before garbage collection could catch up.
00202 |         if (!inFlightRead) {
00203 |             inFlightRead = readSnapshot().finally(() => {
00204 |                 inFlightRead = null;
00205 |             });
00206 |         }
00207 |         return inFlightRead;
00208 |     },
00209 |     update(mutator) {
00210 |         return runUpdates(mutator);
00211 |     },
00212 |     async readMaterialBlob(materialId) {
00213 |         await ensureTable();
00214 |         const result = await pool.query(
00215 |             `SELECT item->'dataBase64' AS bytes
```

```
00216 |         FROM campuspulse_store, jsonb_array_elements(data->'courseMaterials') AS item
00217 |         WHERE id = 1 AND item->>'id' = $1
00218 |         LIMIT 1',
00219 |         [String(materialId)],
00220 |     );
00221 |     return result.rows[0]?.bytes || null;
00222 | },
00223 | type: "postgres",
00224 | };
00225 | }
00226 |
00227 | module.exports = { createPostgresStore };
```

=====

FILE: backend/src/mailer.js

=====

```
00001 | const nodemailer = require("nodemailer");
00002 |
00003 | function escapeHtml(value) {
00004 |     return String(value)
00005 |         .replaceAll("&", "&amp;")
00006 |         .replaceAll("<", "&lt;")
00007 |         .replaceAll(">", "&gt;")
00008 |         .replaceAll("'", "&quot;")
00009 |         .replaceAll('"', "&#039;");
00010 | }
00011 |
00012 | // Providers cap how fast they accept mail. Resend's free plan allows two
00013 | // requests a second, and a class signing up together sails past that: the
00014 | // rejected requests were reported as sent, so those students never received a
00015 | // code. Sends are paced and retried instead.
00016 | const PROVIDER_MIN_INTERVAL_MS = {
00017 |     resend: 600,
00018 |     brevo: 150,
00019 |     smtp: 250,
00020 | };
00021 |
00022 | // A wrong API key or an unverified sender domain will fail the same way however
00023 | // many times it is retried, so those surface immediately.
00024 | const PERMANENT_STATUSES = new Set([400, 401, 403, 404, 405, 413, 422]);
00025 |
00026 | function delay(ms) {
00027 |     return new Promise((resolve) => setTimeout(resolve, ms));
00028 | }
00029 |
00030 | function retryAfterMs(response) {
00031 |     const header = response?.headers?.get?.("retry-after");
00032 |     if (!header) return null;
00033 |     const seconds = Number(header);
00034 |     if (Number.isFinite(seconds)) return Math.max(0, seconds * 1000);
00035 |     const at = Date.parse(header);
00036 |     return Number.isFinite(at) ? Math.max(0, at - Date.now()) : null;
00037 | }
00038 |
00039 | function describe(body) {
00040 |     try {
00041 |         const parsed = JSON.parse(body);
00042 |         return parsed.message || parsed.error || body;
00043 |     } catch {
00044 |         // Not JSON, so the raw body is the clearest description available.
00045 |         return body;
00046 |     }
00047 | }
00048 |
00049 | function deliveryFailure(message) {
00050 |     const failure = new Error(message);
00051 |     // The caller turns this into an explanation rather than a bare 500.
00052 |     failure.deliveryFailed = true;
00053 |     return failure;
00054 | }
00055 |
00056 | /**
00057 |  * Runs sends one at a time, no faster than the provider accepts them.
00058 |  *
00059 |  * Requests queue rather than racing, so a burst of sign-ups is spread out
00060 |  * instead of being partly rejected.
00061 |  */
00062 | function createSendQueue(minIntervalMs) {
00063 |     let chain = Promise.resolve();
00064 |     let lastStartedAt = 0;
00065 |     return function enqueue(task) {
00066 |         const result = chain.then(async () => {
00067 |             const since = Date.now() - lastStartedAt;
00068 |             if (since < minIntervalMs) await delay(minIntervalMs - since);
00069 |             lastStartedAt = Date.now();
00070 |             return task();
00071 |         });
00072 |         chain = result.then(
00073 |             () => {},
00074 |             () => {},
00075 |         );
00076 |         return result;
00077 |     };
```

```
00078 | }
00079 |
00080 | function createMailer(env = process.env) {
00081 |   // Brevo verifies a single sender address, so mail can reach anyone without
00082 |   // owning a domain – the one route that works on a host blocking SMTP.
00083 |   const brevoConfigured = Boolean(env.BREVO_API_KEY && env.EMAIL_FROM);
00084 |   const resendConfigured = Boolean(env.RESEND_API_KEY && env.EMAIL_FROM);
00085 |   const smtpConfigured = Boolean(env.SMTP_HOST && env.SMTP_USER && env.SMTP_PASS);
00086 |   const configured = brevoConfigured || resendConfigured || smtpConfigured;
00087 |   const provider = brevoConfigured
00088 |     ? "brevo"
00089 |     : resendConfigured
00090 |       ? "resend"
00091 |       : smtpConfigured
00092 |         ? "smtp"
00093 |         : "disabled";
00094 |   // Some hosts block outbound SMTP entirely, and a blocked connection would
00095 |   // otherwise hang the request that is waiting on it.
00096 |   const smtpTimeoutMs = Number(env.SMTP_TIMEOUT_MS || 12000);
00097 |   const transporter = smtpConfigured
00098 |     ? nodemailer.createTransport({
00099 |       host: env.SMTP_HOST,
00100 |       port: Number(env.SMTP_PORT || 587),
00101 |       secure: String(env.SMTP_SECURE || "").toLowerCase() === "true",
00102 |       auth: { user: env.SMTP_USER, pass: env.SMTP_PASS },
00103 |       connectionTimeout: smtpTimeoutMs,
00104 |       greetingTimeout: smtpTimeoutMs,
00105 |       socketTimeout: smtpTimeoutMs,
00106 |       // One connection reused for a queued burst beats a fresh handshake per
00107 |       // message, which is what tips a provider into throttling.
00108 |       pool: true,
00109 |       maxConnections: 1,
00110 |       maxMessages: 50,
00111 |     })
00112 |     : null;
00113 |
00114 |   const minIntervalMs = Number(
00115 |     env.EMAIL_MIN_INTERVAL_MS || PROVIDER_MIN_INTERVAL_MS[provider] || 0,
00116 |   );
00117 |   const maxAttempts = Math.max(1, Number(env.EMAIL_MAX_ATTEMPTS || 4));
00118 |   const enqueue = createSendQueue(minIntervalMs);
00119 |
00120 |   // Surfaced through /api/health so a missing code can be diagnosed without
00121 |   // shell access to the server.
00122 |   const stats = {
00123 |     queued: 0,
00124 |     delivered: 0,
00125 |     failed: 0,
00126 |     retried: 0,
00127 |     lastError: null,
00128 |     lastErrorAt: null,
00129 |     lastDeliveredAt: null,
00130 |   };
00131 |
00132 |   async function postJson(url, headers, body) {
00133 |     let lastDetail = "";
00134 |     for (let attempt = 1; attempt <= maxAttempts; attempt += 1) {
00135 |       let response;
00136 |       try {
00137 |         response = await fetch(url, {
00138 |           method: "POST",
00139 |           headers,
00140 |           body: JSON.stringify(body),
00141 |           signal: AbortSignal.timeout(20000),
00142 |         });
00143 |       } catch (error) {
00144 |         lastDetail = String(error?.message || error);
00145 |         if (attempt === maxAttempts) break;
00146 |         stats.retried += 1;
00147 |         await delay(500 * 2 ** (attempt - 1));
00148 |         continue;
00149 |       }
00150 |
00151 |       if (response.ok) return;
00152 |
00153 |       lastDetail = describe(await response.text());
00154 |       if (PERMANENT_STATUSES.has(response.status) || attempt === maxAttempts) break;
00155 |       // 429 and 5xx are worth another try; the provider tells us how long to
00156 |       // wait when it can.
00157 |       stats.retried += 1;
00158 |       await delay(retryAfterMs(response) ?? 500 * 2 ** (attempt - 1));
00159 |     }
00160 |     throw deliveryFailure(`Email provider rejected the request: ${lastDetail}`);
00161 |   }
00162 |
00163 |   async function deliver({ email, name, code, subject, purpose }) {
00164 |     const label = purpose || "verification code";
00165 |     const heading = subject || "Verify your CampusPulse email";
00166 |     const text = `Hello ${name}, your CampusPulse ${label} is ${code}. It expires in 10 minutes.`;
00167 |     const html = `<p>Hello ${escapeHtml(name)},</p><p>Your CampusPulse ${escapeHtml(label)} is
<strong>${code}</strong></p><p>It expires in 10 minutes.</p>`;
00168 |
00169 |     if (brevoConfigured) {
```



```
00170 |     const match = String(env.EMAIL_FROM).match(/^\s*(.*?)\s*<([>]+)>\s*$/);
00171 |     const sender = match
00172 |     ? { name: match[1] || "CampusPulse", email: match[2] }
00173 |     : { name: "CampusPulse", email: String(env.EMAIL_FROM).trim() };
00174 |     await postJson(
00175 |       "https://api.brevo.com/v3/smtp/email",
00176 |       {
00177 |         "api-key": env.BREVO_API_KEY,
00178 |         "Content-Type": "application/json",
00179 |         Accept: "application/json",
00180 |       },
00181 |       {
00182 |         sender,
00183 |         to: [{ email, name }],
00184 |         subject: heading,
00185 |         textContent: text,
00186 |         htmlContent: html,
00187 |       },
00188 |     );
00189 |     return { delivered: true };
00190 |   }
00191 |
00192 |   if (resendConfigured) {
00193 |     await postJson(
00194 |       "https://api.resend.com/emails",
00195 |       {
00196 |         Authorization: `Bearer ${env.RESEND_API_KEY}`,
00197 |         "Content-Type": "application/json",
00198 |       },
00199 |       { from: env.EMAIL_FROM, to: [email], subject: heading, text, html },
00200 |     );
00201 |     return { delivered: true };
00202 |   }
00203 |
00204 |   if (!transporter) {
00205 |     console.log(`[CampusPulse] ${label} for ${email}: ${code}`);
00206 |     return { delivered: false, previewCode: code };
00207 |   }
00208 |
00209 |   let lastReason = "";
00210 |   for (let attempt = 1; attempt <= maxAttempts; attempt += 1) {
00211 |     try {
00212 |       await transporter.sendMail({
00213 |         from: env.SMTP_FROM || env.SMTP_USER,
00214 |         to: email,
00215 |         subject: heading,
00216 |         text,
00217 |         html,
00218 |       });
00219 |       return { delivered: true };
00220 |     } catch (error) {
00221 |       lastReason = String(error?.code || error?.message || "");
00222 |       // A refused or timed-out connection means the host is blocking SMTP,
00223 |       // and no number of retries will change that.
00224 |       if (/ECONNREFUSED|E_SOCKET|E_CONNECTION|Greeting never received/i.test(lastReason)) {
00225 |         throw deliveryFailure(
00226 |           "The mail server could not be reached. Outbound SMTP is often blocked; use an HTTPS provider
00227 |           such as Brevo or Resend instead.",
00228 |         );
00229 |       }
00230 |       // A 4xx SMTP reply is a permanent rejection of this message.
00231 |       if (/^5\d\d/.test(String(error?.responseCode || ""))) break;
00232 |       if (attempt === maxAttempts) break;
00233 |       stats.retried += 1;
00234 |       await delay(500 * 2 ** (attempt - 1));
00235 |     }
00236 |     throw deliveryFailure(`Email could not be sent: ${lastReason}`);
00237 |   }
00238 |
00239 |   return {
00240 |     configured,
00241 |     provider,
00242 |     stats,
00243 |     async sendPasswordReset({ email, name, code }) {
00244 |       return this.sendVerification({
00245 |         email,
00246 |         name,
00247 |         code,
00248 |         subject: "Reset your CampusPulse password",
00249 |         purpose: "password reset code",
00250 |       });
00251 |     },
00252 |     async sendVerification(message) {
00253 |       stats.queued += 1;
00254 |       try {
00255 |         const outcome = await enqueue(() => deliver(message));
00256 |         if (outcome.delivered) {
00257 |           stats.delivered += 1;
00258 |           stats.lastDeliveredAt = new Date().toISOString();
00259 |         }
00260 |         return outcome;
00261 |       } catch (error) {
```

```
00262 |         stats.failed += 1;
00263 |         stats.lastError = String(error?.message || error).slice(0, 300);
00264 |         stats.lastErrorAt = new Date().toISOString();
00265 |         // Named explicitly so an administrator can see exactly who missed a
00266 |         // code rather than inferring it from a complaint.
00267 |         console.error(
00268 |             `[CampusPulse] verification email to ${message?.email} failed: ${stats.lastError}`,
00269 |         );
00270 |         throw error;
00271 |     }
00272 | },
00273 | };
00274 | }
00275 |
00276 | module.exports = { createMailer };
```

=====

FILE: backend/src/push-notifier.js

=====

```
00001 | const crypto = require("node:crypto");
00002 |
00003 | const FIREBASE_SCOPE = "https://www.googleapis.com/auth/firebase.messaging";
00004 | const GOOGLE_TOKEN_URL = "https://oauth2.googleapis.com/token";
00005 |
00006 | function base64Url(value) {
00007 |     return Buffer.from(value).toString("base64url");
00008 | }
00009 |
00010 | // FCM only accepts string values in a data message. Keeping this conversion in
00011 | // the notifier also protects future callers that pass numbers or booleans.
00012 | function normalizeMessageData(value) {
00013 |     if (!value || Array.isArray(value) || typeof value !== "object") return {};
00014 |     return Object.fromEntries(
00015 |         Object.entries(value)
00016 |             .slice(0, 50)
00017 |             .filter(([key, item]) => item !== undefined && item !== null)
00018 |             .map(([key, item]) => {
00019 |                 let normalized;
00020 |                 if (typeof item === "string") normalized = item;
00021 |                 else if (["number", "boolean", "bigint"].includes(typeof item)) {
00022 |                     normalized = String(item);
00023 |                 } else {
00024 |                     try {
00025 |                         normalized = JSON.stringify(item);
00026 |                     } catch {
00027 |                         normalized = String(item);
00028 |                     }
00029 |                 }
00030 |                 return [String(key).slice(0, 100), normalized.slice(0, 4000)];
00031 |             })
00032 |             .filter(([key]) => key),
00033 |     );
00034 | }
00035 |
00036 | function invalidRegistrationToken(status, payload) {
00037 |     const error = payload?.error || {};
00038 |     const details = Array.isArray(error.details) ? error.details : [];
00039 |     if (details.some((detail) => detail?.errorCode === "UNREGISTERED")) return true;
00040 |     const message = String(error.message || "");
00041 |     return (
00042 |         (status === 400 || status === 404) &&
00043 |         /registration token|requested entity was not found/i.test(message)
00044 |     );
00045 | }
00046 |
00047 | function disabledNotifier(status = "disabled") {
00048 |     return {
00049 |         configured: false,
00050 |         provider: "firebase-http-v1",
00051 |         status,
00052 |         async send() {
00053 |             return { delivered: false, reason: status };
00054 |         },
00055 |     };
00056 | }
00057 |
00058 | function createFirebaseNotifier(env = process.env, options = {}) {
00059 |     const raw = String(env.FIREBASE_SERVICE_ACCOUNT_JSON || "").trim();
00060 |     if (!raw) return disabledNotifier();
00061 |
00062 |     let serviceAccount;
00063 |     try {
00064 |         serviceAccount = JSON.parse(raw);
00065 |     } catch {
00066 |         return disabledNotifier("invalid");
00067 |     }
00068 |
00069 |     const projectId = String(serviceAccount?.project_id || "").trim();
00070 |     const clientEmail = String(serviceAccount?.client_email || "").trim();
00071 |     const privateKey = String(serviceAccount?.private_key || "").replace(/\n/g, "\\n");
00072 |     if (!projectId || !clientEmail || !privateKey) return disabledNotifier("invalid");
00073 |
00074 |     let signingKey;
```

```
00075 |     try {
00076 |         signingKey = crypto.createPrivateKey(privateKey);
00077 |     } catch {
00078 |         return disabledNotifier("invalid");
00079 |     }
00080 |
00081 |     const fetchImpl = options.fetch || globalThis.fetch;
00082 |     if (typeof fetchImpl !== "function") return disabledNotifier("unavailable");
00083 |     const timeoutMs = Math.max(1000, Number(options.timeoutMs) || 10_000);
00084 |     const requestSignal = () =>
00085 |         typeof globalThis.AbortSignal?.timeout === "function"
00086 |             ? globalThis.AbortSignal.timeout(timeoutMs)
00087 |             : undefined;
00088 |
00089 |     let accessToken = "";
00090 |     let accessTokenExpiresAt = 0;
00091 |     let accessTokenRequest = null;
00092 |
00093 |     async function getAccessToken() {
00094 |         if (accessToken && accessTokenExpiresAt > Date.now() + 60_000) return accessToken;
00095 |         if (!accessTokenRequest) {
00096 |             accessTokenRequest = (async () => {
00097 |                 const issuedAt = Math.floor(Date.now() / 1000);
00098 |                 const header = base64Url(JSON.stringify({ alg: "RS256", typ: "JWT" }));
00099 |                 const claims = base64Url(
00100 |                     JSON.stringify({
00101 |                         iss: clientEmail,
00102 |                         scope: FIREBASE_SCOPE,
00103 |                         aud: GOOGLE_TOKEN_URL,
00104 |                         iat: issuedAt,
00105 |                         exp: issuedAt + 3600,
00106 |                     }),
00107 |                 );
00108 |                 const unsignedJwt = `${header}.${claims}`;
00109 |                 const signature = crypto
00110 |                     .sign("RSA-SHA256", Buffer.from(unsignedJwt), signingKey)
00111 |                     .toString("base64url");
00112 |                 const assertion = `${unsignedJwt}.${signature}`;
00113 |                 const response = await fetchImpl(GOOGLE_TOKEN_URL, {
00114 |                     method: "POST",
00115 |                     headers: { "content-type": "application/x-www-form-urlencoded" },
00116 |                     signal: requestSignal(),
00117 |                     body: new URLSearchParams({
00118 |                         grant_type: "urn:ietf:params:oauth:grant-type:jwt-bearer",
00119 |                         assertion,
00120 |                     }),
00121 |                 });
00122 |                 const payload = await response.json().catch(() => ({}));
00123 |                 if (!response.ok || !payload.access_token) {
00124 |                     const error = new Error("Firebase authentication failed");
00125 |                     error.status = response.status;
00126 |                     throw error;
00127 |                 }
00128 |                 accessToken = String(payload.access_token);
00129 |                 const expiresIn = Math.max(60, Number(payload.expires_in) || 3600);
00130 |                 accessTokenExpiresAt = Date.now() + expiresIn * 1000;
00131 |                 return accessToken;
00132 |             })();
00133 |         }
00134 |         try {
00135 |             return await accessTokenRequest;
00136 |         } finally {
00137 |             accessTokenRequest = null;
00138 |         }
00139 |     }
00140 |
00141 |     return {
00142 |         configured: true,
00143 |         provider: "firebase-http-v1",
00144 |         status: "configured",
00145 |         async send({ token, title, body = "", data = {} }) {
00146 |             const bearerToken = await getAccessToken();
00147 |             const response = await fetchImpl(
00148 |                 `https://fcm.googleapis.com/v1/projects/${encodeURIComponent(projectId)}/messages:send`,
00149 |                 {
00150 |                     method: "POST",
00151 |                     headers: {
00152 |                         authorization: `Bearer ${bearerToken}`,
00153 |                         "content-type": "application/json",
00154 |                     },
00155 |                     signal: requestSignal(),
00156 |                     body: JSON.stringify({
00157 |                         message: {
00158 |                             token: String(token),
00159 |                             notification: {
00160 |                                 title: String(title || "CampusPulse").slice(0, 120),
00161 |                                 body: String(body || "").slice(0, 500),
00162 |                             },
00163 |                             data: normalizeMessageData(data),
00164 |                             android: {
00165 |                                 priority: "high",
00166 |                                 notification: {
00167 |                                     channel_id: "campuspulse_events",
```

```
00168 |         sound: "default",
00169 |     },
00170 | },
00171 |     apns: { payload: { aps: { sound: "default" } } },
00172 | },
00173 | }},
00174 | },
00175 | );
00176 | const payload = await response.json().catch(() => ({}));
00177 | if (!response.ok) {
00178 |     const error = new Error("Firebase push delivery failed");
00179 |     error.status = response.status;
00180 |     error.invalidToken = invalidRegistrationToken(response.status, payload);
00181 |     throw error;
00182 | }
00183 | return { delivered: true, id: payload.name || "" };
00184 | },
00185 | };
00186 | }
00187 |
00188 | module.exports = {
00189 |     createFirebaseNotifier,
00190 |     invalidRegistrationToken,
00191 |     normalizeMessageData,
00192 | };
```

=====

FILE: backend/src/maintenance.js

=====

```
00001 | // Bumping this id replays the reset once on the next start.
00002 | const ACCOUNT_RESET_ID = "reset-to-empty-workspace-2026-08-02";
00003 |
00004 | async function deleteExistingAccountsOnce(store) {
00005 |     // Once the migration marker exists, avoid opening a write transaction on
00006 |     // every boot. PostgreSQL writes load and stringify the full shared document,
00007 |     // including material blobs, which creates a large and needless memory spike
00008 |     // while a small instance is recovering from a restart.
00009 |     const snapshot = await store.read();
00010 |     if (snapshot.maintenance.includes(ACCOUNT_RESET_ID)) {
00011 |         return { applied: false, deletedAccounts: 0, deletedCourses: 0 };
00012 |     }
00013 |     return store.update((database) => {
00014 |         if (database.maintenance.includes(ACCOUNT_RESET_ID)) {
00015 |             return { applied: false, deletedAccounts: 0, deletedCourses: 0 };
00016 |         }
00017 |
00018 |         const deletedAccounts = database.users.length;
00019 |         const deletedCourses = database.courses.length;
00020 |
00021 |         // CampusPulse starts empty: every course, roll list, attendance record and
00022 |         // quiz belongs to a professor who now has to create it again.
00023 |         database.users = [];
00024 |         database.verificationCodes = [];
00025 |         database.sessions = [];
00026 |         database.enrollments = [];
00027 |         database.courses = [];
00028 |         database.courseStudents = [];
00029 |         database.courseMaterials = [];
00030 |         database.courseNotices = [];
00031 |         database.notifications = [];
00032 |         database.pushDevices = [];
00033 |         database.schedule = [];
00034 |         database.attendanceSessions = [];
00035 |         database.quizzes = [];
00036 |
00037 |         database.maintenance.push(ACCOUNT_RESET_ID);
00038 |         return { applied: true, deletedAccounts, deletedCourses };
00039 |     });
00040 | }
00041 |
00042 | // Clears attendance and quiz data only – courses, students, schedules stay.
00043 | const ATTENDANCE QUIZ_RESET_ID = "clear-attendance-quiz-2026-08-02";
00044 |
00045 | async function clearAttendanceAndQuizzesOnce(store) {
00046 |     const snapshot = await store.read();
00047 |     if (snapshot.maintenance.includes(ATTENDANCE QUIZ_RESET_ID)) {
00048 |         return { applied: false };
00049 |     }
00050 |     return store.update((database) => {
00051 |         if (database.maintenance.includes(ATTENDANCE QUIZ_RESET_ID)) {
00052 |             return { applied: false };
00053 |         }
00054 |
00055 |         const deletedSessions = database.attendanceSessions.length;
00056 |         const deletedQuizzes = database.quizzes.length;
00057 |
00058 |         database.attendanceSessions = [];
00059 |         database.quizzes = [];
00060 |
00061 |         database.maintenance.push(ATTENDANCE QUIZ_RESET_ID);
00062 |         return { applied: true, deletedSessions, deletedQuizzes };
00063 |     });
00064 | }
```

```
00065 |
00066 | module.exports = {
00067 |   ACCOUNT_RESET_ID,
00068 |   deleteExistingAccountsOnce,
00069 |   ATTENDANCE QUIZ_RESET_ID,
00070 |   clearAttendanceAndQuizzesOnce,
00071 | };
```

FILE: backend/src/profile-overrides.js

```
00001 | function professorProfileOverrides(env = process.env) {
00002 |   const raw = String(env.PROFESSOR_PROFILE_OVERRIDES_JSON || "").trim();
00003 |   if (!raw) return new Map();
00004 |   let parsed;
00005 |   try {
00006 |     parsed = JSON.parse(raw);
00007 |   } catch {
00008 |     throw new Error("PROFESSOR_PROFILE_OVERRIDES_JSON must be valid JSON");
00009 |   }
00010 |   if (!parsed || Array.isArray(parsed) || typeof parsed !== "object") {
00011 |     throw new Error("PROFESSOR_PROFILE_OVERRIDES_JSON must be a JSON object");
00012 |   }
00013 |   return new Map(
00014 |     Object.entries(parsed).map(([email, profile]) => [
00015 |       String(email).trim().toLowerCase(),
00016 |       {
00017 |         ...(String(profile?.phone || "").trim()
00018 |           ? { phone: String(profile.phone).trim() }
00019 |           : {}),
00020 |         ...(String(profile?.department || "").trim()
00021 |           ? { department: String(profile.department).trim() }
00022 |           : {}),
00023 |       },
00024 |     ]),
00025 |   );
00026 | }
00027 |
00028 | function applyUserProfileOverride(user, env = process.env) {
00029 |   if (!user || user.role !== "faculty") return user;
00030 |   const override = professorProfileOverrides(env).get(
00031 |     String(user.email || "").trim().toLowerCase(),
00032 |   );
00033 |   return override ? { ...user, ...override } : user;
00034 | }
00035 |
00036 | async function applyProfessorProfileOverrides(store, env = process.env) {
00037 |   const overrides = professorProfileOverrides(env);
00038 |   if (!overrides.size) return { updated: 0 };
00039 |   const snapshot = await store.read();
00040 |   const needsUpdate = snapshot.users.some((user) => {
00041 |     if (user.role !== "faculty") return false;
00042 |     const override = overrides.get(String(user.email || "").trim().toLowerCase());
00043 |     return Boolean(
00044 |       override &&
00045 |       (override.phone !== undefined && override.phone !== user.phone ||
00046 |         override.department !== undefined && override.department !== user.department),
00047 |     );
00048 |   });
00049 |   if (!needsUpdate) return { updated: 0 };
00050 |   return store.update((database) => {
00051 |     let updated = 0;
00052 |     database.users = database.users.map((user) => {
00053 |       if (user.role !== "faculty") return user;
00054 |       const override = overrides.get(String(user.email || "").trim().toLowerCase());
00055 |       if (!override) return user;
00056 |       const overridden = { ...user, ...override };
00057 |       if (
00058 |         overridden.phone !== user.phone ||
00059 |         overridden.department !== user.department
00060 |       ) {
00061 |         updated += 1;
00062 |       }
00063 |       return overridden;
00064 |     });
00065 |     return { updated };
00066 |   });
00067 | }
00068 |
00069 | module.exports = {
00070 |   professorProfileOverrides,
00071 |   applyProfessorProfileOverrides,
00072 |   applyUserProfileOverride,
00073 | };
```

FILE: backend/src/course-data.js

```
00001 | // CampusPulse ships with no courses. A professor creates each course and
00002 | // uploads its roll list, so there is nothing to seed from configuration.
00003 | function buildCourseData() {
00004 |   return { courses: [], courseStudents: [] };
00005 | }
```

```
00006 |
00007 | module.exports = { buildCourseData };
```

FILE: backend/src/security.js

```
00001 | const crypto = require("node:crypto");
00002 | const { promisify } = require("node:util");
00003 |
00004 | const scrypt = promisify(crypto.scrypt);
00005 |
00006 | async function hashPassword(password, salt = crypto.randomBytes(16).toString("hex")) {
00007 |   const derivedKey = await scrypt(password, salt, 64);
00008 |   return `${salt}:${derivedKey.toString("hex")}`;
00009 | }
00010 |
00011 | async function verifyPassword(password, storedHash) {
00012 |   const [salt, expectedHex] = String(storedHash || "").split(":");
00013 |   if (!salt || !expectedHex) return false;
00014 |   const actual = await scrypt(password, salt, 64);
00015 |   const expected = Buffer.from(expectedHex, "hex");
00016 |   return expected.length === actual.length && crypto.timingSafeEqual(expected, actual);
00017 | }
00018 |
00019 | function randomCode() {
00020 |   return String(crypto.randomInt(100000, 1000000));
00021 | }
00022 |
00023 | function randomToken() {
00024 |   return crypto.randomBytes(32).toString("base64url");
00025 | }
00026 |
00027 | function sha256(value) {
00028 |   return crypto.createHash("sha256").update(String(value)).digest("hex");
00029 | }
00030 |
00031 | module.exports = { hashPassword, verifyPassword, randomCode, randomToken, sha256 };
```

FILE: backend/src/server.js

```
00001 | // Password hashing is deliberately limited to four concurrent libuv workers.
00002 | // Each scrypt job reserves a sizeable native-memory buffer, so widening this
00003 | // pool to sixteen can exhaust a small Render instance during a login burst.
00004 | // Requests may queue briefly, but the process stays alive to serve them.
00005 | if (!process.env.UV_THREADPOOL_SIZE) {
00006 |   process.env.UV_THREADPOOL_SIZE = "4";
00007 | }
00008 |
00009 | const path = require("node:path");
00010 | const { createApp } = require("./app");
00011 | const { deleteExistingAccountsOnce, clearAttendanceAndQuizzesOnce } = require("./maintenance");
00012 | const { applyProfessorProfileOverrides } = require("./profile-overrides");
00013 |
00014 | const port = Number(process.env.PORT || 8787);
00015 | const host = process.env.HOST || "0.0.0.0";
00016 | const databasePath =
00017 |   process.env.DATABASE_PATH ||
00018 |   path.resolve(__dirname, "../data/campuspulse.json");
00019 | const { app, store } = createApp({ databasePath });
00020 |
00021 | async function start() {
00022 |   const reset = await deleteExistingAccountsOnce(store);
00023 |   if (reset.applied) {
00024 |     console.log(`CampusPulse account reset: deleted ${reset.deletedAccounts} account(s)`);
00025 |   }
00026 |   const aqReset = await clearAttendanceAndQuizzesOnce(store);
00027 |   if (aqReset.applied) {
00028 |     console.log(`CampusPulse data reset: cleared ${aqReset.deletedSessions} attendance session(s) and
00029 | ${aqReset.deletedQuizzes} quiz(zes)`);
00030 |   }
00031 |   const profileOverrides = await applyProfessorProfileOverrides(store, process.env);
00032 |   if (profileOverrides.updated) {
00033 |     console.log(
00034 |       `CampusPulse professor profiles updated: ${profileOverrides.updated} account(s)`,
00035 |     );
00036 |   }
00037 |   const server = app.listen(port, host, () => {
00038 |     console.log(`CampusPulse API listening on http://${host}:${port}`);
00039 |     console.log(
00040 |       `CampusPulse database: ${process.env.TARGET_DATABASE_URL || process.env.DATABASE_URL ?
00041 | "PostgreSQL" : databasePath}`,
00042 |     );
00043 |   });
00044 |   server.headersTimeout = 65000;
00045 |   server.requestTimeout = 60000;
00046 |   server.keepAliveTimeout = 30000;
00047 | }
00048 | start().catch((error) => {
00049 |   console.error("CampusPulse failed to start", error);
00050 |   process.exitCode = 1;
00051 | });
```